LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING

Preinstall to persistence: Inside the Red Hat npm Miasma credential-stealing campaign

Malware Analysis

Classification	TLP:CLEAR
Published	2026-06-03
Version	1.0
Author	Lost Boys Cyber
Report Type	Malware Analysis

Published: 2026-06-03 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

Preinstall to Persistence: Inside the Red Hat npm Miasma Credential-Stealing Campaign

1. Executive Summary

Miasma is a credential-stealing, self-propagating npm supply-chain campaign that abused legitimate `@redhat-cloud-services` packages as trusted delivery vehicles. Microsoft Threat Intelligence reported the campaign as a large-scale npm supply-chain attack affecting more than 90 versions of Red Hat Cloud Services packages; JFrog Security Research separately analyzed 96 hijacked package versions, including `@redhat-cloud-services/types` version 3.6.1. The campaign is not a typosquatting event: defenders should treat affected installs as exposure through legitimate dependency names, package provenance, and CI/CD trust relationships.

The malware executes at install time, before application code imports the package. In the Red Hat package wave, a weaponized `preinstall` script runs `node index.js`; related Miasma/Shai-Hulud variants also abuse `binding.gyp` command expansion to execute without obvious lifecycle script entries. The loader unwraps a layered JavaScript payload, installs or invokes the Bun runtime, runs transient payloads under `/tmp`, collects developer and cloud credentials, exfiltrates via GitHub dead-drop repositories and traffic shaped to look like Anthropic API activity, and attempts worm-like propagation by republishing packages using stolen npm tokens or GitHub Actions OIDC identities.

The highest business risk is credential and build-chain compromise, not only malware execution on a single workstation. Affected developer machines and CI runners may expose GitHub tokens, npm publish permissions, cloud IAM credentials, Vault tokens, Kubernetes service account tokens, SSH keys, `.env` secrets, and package-release workflows. The malware also includes persistence and remediation-trap behavior: it can add developer-tool hooks, VS Code folder-open tasks, user services, LaunchAgents, and a GitHub-token monitor that may execute destructive commands after a token is invalidated. Incident response should isolate affected systems before token revocation.

Lost Boys Cyber assesses this campaign as high-confidence supply-chain malware with broad defender relevance for organizations using npm, GitHub Actions trusted publishing, cloud build infrastructure, or AI-assisted development tooling. The immediate action is to inventory affected package versions, isolate exposed runners and developer endpoints, preserve evidence, remove persistence, rotate secrets from a clean environment, and hunt for install-time execution, unexpected Bun activity, GitHub dead-drop artifacts, and package-publishing anomalies.

Finding	Defensive Implication	Priority
Legitimate Red Hat npm namespace abused	Dependency name trust and provenance alone are insufficient	Critical
Install-time execution via `preinstall`; related variants via `binding.gyp`	CI/CD and endpoint detections must watch package installation behavior, not only application runtime	Critical
Credential collection spans GitHub, npm, AWS, Azure, GCP, Vault, Kubernetes, SSH, Docker, and local files	Treat exposed build systems as identity-compromise events	Critical
GitHub/npm worm propagation and OIDC trusted-publishing abuse	Review package releases, workflow changes, signed commits, and provenance attestations for malicious but valid-looking output	High
Developer-tool and dead-man-switch persistence	Do not revoke tokens before isolating and cleaning infected hosts	High

2. Sample Metadata

Field	Value
Campaign / Family	Miasma: The Spreading Blight; Shai-Hulud / Mini Shai-Hulud variant
Malware Type	Credential stealer, npm supply-chain worm, CI/CD compromise enabler
Platform	npm packages; Node.js and Bun execution on Linux, macOS, Windows, developer endpoints, and CI runners
Primary Analyzed Package	`@redhat-cloud-services/types` version 3.6.1, per JFrog analysis
Primary Package Metadata SHA256	`7069e28a5806db4ab0273639667d203f5e31b401d403af7e36d9f360c1f6d655`
Obfuscated Loader SHA256	`b86c5ae9e95bd841a595440faa3eb6317441e746f241ae8fd641ab59ed1d1966`
Affected Scope	Microsoft: 32 maliciously modified packages across more than 90 versions; JFrog: 96 hijacked `@redhat-cloud-services` package versions
Initial Public Reporting	2026-06-01 to 2026-06-03 reporting window; Microsoft article published 2026-06-03 in RSS metadata
Delivery Theme	Legitimate Red Hat Cloud Services frontend/client packages with install-time malware
Primary Objective	Steal credentials and propagate through packages/repositories controlled by compromised identities
Analyst Confidence	High for behavior and affected namespace; Medium for attribution beyond Shai-Hulud/TeamPCP tooling lineage
Local Detonation Status	Vendor-report and open-source-research based; no local detonation performed for this intake report

Representative Affected Red Hat Package	Reported Malicious Versions	Notes
`@redhat-cloud-services/types`	3.6.1, 3.6.2, 3.6.4	Type-only package with suspicious `preinstall` execution in analyzed sample
`@redhat-cloud-services/frontend-components`	7.7.2, 7.7.3, 7.7.5	High-risk dependency due to frontend component reuse
`@redhat-cloud-services/rbac-client`	9.0.3, 9.0.4, 9.0.6	Identity/authorization client package name increases defender concern
`@redhat-cloud-services/sources-client`	3.0.10, 3.0.11, 3.0.13	Generated API client family
`@redhat-cloud-services/vulnerabilities-client`	2.1.8, 2.1.9, 2.1.11	Security-related client package, affected per public IOC lists

3. Source Review and Confidence

Source	Contribution	Confidence	Notes
Microsoft Threat Intelligence, “Preinstall to persistence…”	Primary task source; campaign framing, affected scale, attack-chain overview, mitigation guidance	High	Microsoft page was unavailable to direct extraction during this run due access throttling, but RSS metadata and search snippets corroborate title, summary, and key claims
JFrog Security Research, “Shai-Hulud - Miasma…”	Detailed loader, staging, package list, hashes, IOCs, persistence, propagation, and remediation	High	Direct page retrieval and text extraction succeeded locally
Cloud Security Alliance research note	Cross-source synthesis covering Red Hat npm compromise, OIDC trusted publishing, credential targets, mitigations	High	PDF downloaded and converted to text locally
The Hacker News summary with vendor cross-references	Secondary corroboration of credential classes, dead-drop behavior, persistence artifacts, and response order	Medium	Useful as corroboration, not treated as primary technical evidence
StepSecurity reporting on Phantom Gyp	Related Miasma variant behavior and binding.gyp execution method	Medium	Applies directly to family-level detection and hunt coverage; Red Hat wave primarily used explicit `preinstall` hooks

Observed facts are the affected npm namespace, install-time execution path, Red Hat package list, JFrog-provided hashes, and named host/repository artifacts. High-confidence assessment is that Miasma should be handled as a supply-chain identity compromise event because the payload targets credentials and package-publishing authority. Lower-confidence assessment is actor attribution: public reporting ties the malware lineage to Shai-Hulud / Mini Shai-Hulud and notes TeamPCP-related tooling, but copycat use of released code cannot be ruled out.

This report intentionally avoids overclaiming direct local malware behavior. No local detonation, packet capture, or reverse engineering was performed for this intake card; static/code/dynamic observations below are sourced from Microsoft, JFrog, CSA, StepSecurity, and other public technical reporting.

4. Infection Chain Analysis

4.1. Delivery and Trust Abuse

Miasma entered environments through legitimate package names under `@redhat-cloud-services`. This matters operationally because allowlists, package provenance, and developer familiarity may all favor installation. A transitive dependency can trigger execution during `npm install`, including in ephemeral CI runners where credentials are intentionally available.

Chain Stage	Observed / Reported Behavior	Defensive Meaning
Package compromise	Malicious versions published under legitimate `@redhat-cloud-services` scope	Treat as trusted-namespace compromise, not typo/package confusion
Install trigger	`package.json` `preinstall` executes `node index.js`; family variants also abuse `binding.gyp`	Monitor dependency install behavior and package metadata diffs
Loader execution	Obfuscated JavaScript	Static scanners must unpack or emulate

	reconstructs/decrypts later stages	layered JavaScript
Runtime staging	Bun bootstrapper and main payload written under `/tmp` and executed with Bun	Hunt for `node -> curl/unzip -> bun` or `node -> bun run /tmp/p*.js` chains during npm install
Collection	Environment, local secret files, GitHub/npm/cloud/Kubernetes/Vault/SSH material	Scope as credential theft and cloud identity compromise
Exfiltration	GitHub dead-drop repositories and Anthropic-shaped traffic to `api.anthropic.com/v1/api`	Hunt for full path/request shape and suspicious GitHub repo creation
Propagation	npm package tarball mutation, patch-version bump, republish; GitHub Actions/workflow manipulation	Review releases, workflow commits, provenance attestations, and package ownership changes
Persistence / trap	AI-tool hooks, VS Code tasks, user services, token monitor	Clean endpoints before revoking tokens; deleting `node_modules` is insufficient

4.2. Process Tree and Execution Chain

Representative process lineage for the Red Hat npm wave is below. Exact parent names differ by package manager, shell, operating system, and CI runner, but the chain captures the core telemetry defenders should expect.

Step	Parent Process	Child / Action	Why It Matters
1	`npm`, `pnpm`, `yarn`, or CI package-install step	Reads malicious package metadata	Initial execution may be transitive and not initiated by a user directly
2	Package lifecycle handler	`node index.js` from package root	Type/client packages rarely need install-time script execution
3	`node`	Deobfuscates numeric-array/ROT wrapper and decrypts AES-128-GCM blobs	Layered obfuscation reduces static visibility
4	`node`	Writes `/tmp/b*.js` and `/tmp/p*.js`; downloads Bun if absent	Temporary payload and runtime staging are high-value host indicators
5	`node` / shell	`curl` or equivalent download from Bun release infrastructure, `unzip -j -o b.zip`	Legitimate Bun download during npm install is suspicious in many environments
6	`bun`	`bun run /tmp/p*.js` or `bun run index.js`	Main credential stealer/worm execution
7	Payload	Reads env vars, token stores, metadata services, cloud APIs, repo/workflow data	Identity and supply-chain compromise phase
8	Payload / API clients	Creates GitHub repositories, commits `results/results-*.json`, modifies workflows, publishes	Exfiltration and propagation phase

		npm packages	
9	Payload	Adds user services, LaunchAgents, AI-tool hooks, `.vscode/tasks.json`, `gh-token-monitor`	Persistence and remediation-trap phase

4.3. Exposure Scenarios

- Developer endpoint exposure: A developer installs or updates an affected package locally. The malware can read local shell histories, `.env` files, GitHub CLI tokens, SSH keys, Docker config, cloud credentials, and AI-tool settings.
- CI/CD runner exposure: A build workflow installs a malicious dependency while GitHub Actions tokens, npm trusted-publishing OIDC context, cloud deployment credentials, or secrets are available. The malware can use short-lived but powerful identities within their validity window.
- Downstream propagation: A stolen npm token or trusted-publishing identity allows the payload to republish packages owned by the victim, extending compromise to dependent projects even after the original Red Hat package versions are removed.
- Developer-tool persistence: Payload copies and hooks in `.claude`, `.vscode`, `.github`, `.cursor`, `.gemini`, and related files can re-execute when a project or AI coding assistant session starts.

5. Technical Analysis

5.1. Static and Package Analysis

The analyzed Red Hat sample used package metadata inconsistent with normal package behavior. JFrog documented `@redhat-cloud-services/types` version 3.6.1 as a type package whose `package.json` includes:

```
{
  "name": "@redhat-cloud-services/types",
  "version": "3.6.1",
  "scripts": {
    "preinstall": "node index.js"
  }
}
```

A type-only package executing a JavaScript installer before installation completes is a strong static signal. Red Hat package versions in the campaign reportedly replaced or added a large obfuscated `index.js` loader. Public reporting described a 4.29 MB preinstall dropper in the Microsoft-linked campaign and a single-line obfuscated script replacing legitimate package entrypoints.

Static Signal	Evidence	Analyst Value
Suspicious lifecycle hook	`scripts.preinstall = "node index.js"`	Immediate install-time execution before code import
Type/client package executing installer	`@redhat-cloud-services/types` sample	Strong anomaly for dependency governance
Large obfuscated JavaScript loader	Numeric array, ROT-style transform, eval, AES-128-GCM staging	Requires unpacking; simple string matching may fail
Bun dependency / runtime install	`bun` dependency and Bun bootstrapper	Shifts execution from Node to Bun, bypassing Node-focused telemetry
Temporary payload paths	`/tmp/p*.js`, `/tmp/b*.js`, `/tmp/b-*/bun`	Host hunting indicators for transient execution
Campaign marker	`Miasma: The Spreading Blight`	Useful GitHub dead-drop and repo-description hunt term

5.2. Code Analysis

The loader uses multiple stages designed to slow static analysis and hide the final payload until runtime. JFrog's extracted code path shows the wrapper reconstructing JavaScript from character arrays, decrypting two AES-128-GCM blobs, writing a Bun bootstrapper and the main payload to temporary files, executing the payload with Bun, and deleting the transient file.

Representative logic from public analysis:

```
const bunBootstrap = decrypt(...);
const mainPayload = decrypt(...);
const payloadPath = `/tmp/p${Math.random().toString(36).slice(2)}.js`;
fs.writeFileSync(payloadPath, mainPayload);
if (typeof Bun !== "undefined") {
  execSync(`bun run "${payloadPath}"`, { stdio: "inherit" });
} else {
  await eval(bunBootstrap);
  execSync(`"${getBunPath()}" run "${payloadPath}"`, { stdio: "inherit" });
}
fs.unlinkSync(payloadPath);
```

Routine / Logic	Evidence	Why It Matters
Numeric-array wrapper and ROT transform	JFrog loader analysis	Hides second-stage code from naive scans
AES-128-GCM payload decryption	JFrog and CSA analysis	Allows unique or runtime-resolved payloads
Bun bootstrap	Downloads Bun v1.3.13 builds where absent	Introduces new runtime and process telemetry
Credential regexes	`gh[op]_`, `github_pat_`, `npm_`, GitHub Actions JWT patterns	Enables broad token harvesting across local and CI contexts
GitHub dead-drop creation	Repos with `Miasma: The Spreading Blight` descriptions and `results/results-*.json`	Exfiltration can occur through apparently legitimate GitHub API usage
npm propagation	Tarball download, preinstall injection, patch bump, republish	Converts credential theft into worm behavior
Trusted-publishing abuse	GitHub Actions OIDC token exchange for npm registry	Valid-looking provenance can accompany malicious packages
Token monitor	`IfYouInvalidateThisTokenItWillNukeTheComputerOfTheOwner` marker and gh-token monitor	Revocation-first response can trigger destructive behavior

5.3. Dynamic Behavior and System Modification

Public sandbox/research reporting indicates the malware performs host discovery, credential collection, network calls, package/repository mutations, and persistence. No local detonation was performed during this report; dynamic findings below are source-attributed.

Capability	Supporting Dynamic Evidence
Environment and local secret collection	Reads environment variables, shell history, `.env` files, Git credentials, GitHub CLI tokens, SSH keys, Docker credentials, and token-like strings
Cloud credential collection	Queries AWS IMDS/ECS metadata, AWS Secrets Manager/SSM, Azure IMDS/Graph/Key Vault, GCP metadata/Secret

	Manager/Resource Manager
Kubernetes and Vault collection	Reads Kubernetes service-account paths and probes Vault endpoints such as localhost:8200
Endpoint and CI awareness	Checks for CrowdStrike, SentinelOne, Carbon Black, and StepSecurity Harden-Runner before sensitive operations
Exfiltration	Uses GitHub repositories as dead drops; sends Anthropic-shaped traffic to `api.anthropic.com:443/v1/api` with spoofed `python-requests/2.31.0` user-agent in CSA reporting
npm/GitHub propagation	Republishes packages, injects workflows/action payloads, commits verified-looking workflow changes
Persistence	Installs user-level services/LaunchAgents, AI-tool hooks, VS Code folder-open tasks, and GitHub token monitor

5.4. Persistence and Remediation Traps

Miasma includes persistence beyond package installation. JFrog identified Linux `kitty-monitor.service`, macOS `com.user.kitty-monitor.plist`, `.claude/settings.json`, `.claude/setup.mjs`, `.vscode/tasks.json`, `.github/setup.js`, `~/.config/index.js`, and GitHub token monitor artifacts. Related Hades waves expanded AI-tool targeting to Cursor, Gemini, Windsurf, Copilot, Aider, MCP configs, and additional rule files.

The dead-man-switch behavior is especially important for responders. The token monitor can store a stolen GitHub token and handler, poll GitHub, and execute destructive commands if the token is invalidated. The correct sequence is:

1. Isolate affected endpoint or CI runner from the network.
2. Snapshot/preserve evidence and capture suspicious files, service definitions, and package-lock metadata.
3. Remove persistence and stop token-monitor services from a controlled shell.
4. Revoke and rotate tokens from a clean administrative workstation.
5. Review package releases, GitHub repositories, workflows, and cloud audit logs for follow-on abuse.

6. Indicators of Compromise

The IOC list below prioritizes stable, public, and behaviorally useful indicators. Full package names and versions are preserved in `iocs.csv` in the analyst-kit source bundle.

IOC Type	Value / Pattern	Source	Operational Notes
SHA256	`7069e28a5806db4ab0273639667d203f5e31b401d403af7e36d9f360c1f6d655`	JFrog	Malicious package metadata for `@redhat-cloud-services/types` 3.6.1
SHA256	`b86c5ae9e95bd841a595440faa3eb6317441e746f241ae8fd641ab59ed1d1966`	JFrog	Obfuscated install-time JavaScript loader
Package scope	`@redhat-cloud-services/*` malicious versions	Microsoft/JFrog/CSA	Review lockfiles and artifact caches around 2026-06-01
Package/version	`@redhat-cloud-services/types` 3.6.1, 3.6.2, 3.6.4	JFrog	Primary analyzed package family
Package/version	`@redhat-cloud-services/frontend-components` 7.7.2,	JFrog	Representative high-use

	7.7.3, 7.7.5		frontend package
Network path	`hxxps://api[.]anthropic[.]com/v1/api`	JFrog/Microsoft/CSA	Hunt full path and unexpected client/process context; do not treat whole domain as malicious by itself
GitHub repo description	`Miasma: The Spreading Blight`	JFrog/StepSecurity	Attacker-created or victim-created dead-drop repo marker
GitHub result path	`results/results-*.json`	JFrog	Exfiltrated result files in dead-drop repos
Threat marker	`IfYouInvalidateThisTokenItWillNukeTheComputerOfTheOwner`	JFrog/Socket via THN	Dead-man-switch token marker
Host path	`/tmp/p*.js`, `/tmp/b*.js`, `/tmp/b-*/bun`, `/tmp/b-*/b.zip`, `/tmp/.bun_ran`	JFrog	Transient loader/runtime artifacts
Persistence path	`~/.config/systemd/user/kitty-monitor.service`	JFrog	Linux user service
Persistence path	`~/Library/LaunchAgents/com.user.kitty-monitor.plist`	JFrog	macOS LaunchAgent
Persistence path	`~/.config/gh-token-monitor/token`, `~/.config/gh-token-monitor/handler`, `~/.local/bin/gh-token-monitor.sh`	JFrog	Dead-man-switch monitor artifacts
Developer-tool hook	`.claude/settings.json`, `.claude/setup.mjs`, `.vscode/tasks.json`, `.github/setup.js`, `~/.config/index.js`	JFrog/THN	Re-execution through AI/developer tools
Suspicious command	`node index.js`, `bun run /tmp/p*.js`, `curl -sSL ...bun-v1.3.13`, `unzip -j -o b.zip`, `gh auth token`	JFrog	Process-hunting strings

7. MITRE ATT&CK Mapping

Technique	Name	Evidence
T1195.002	Supply Chain Compromise: Compromise Software Supply Chain	Legitimate `@redhat-cloud-services` npm packages were modified and republished with malicious install-time code
T1059.007	Command and Scripting Interpreter: JavaScript	`node index.js` preinstall execution and JavaScript loader/payload chain
T1059.004	Command and Scripting Interpreter: Unix	Shell commands used for Bun installation,

	Shell	persistence setup, and destructive handlers
T1027	Obfuscated Files or Information	Numeric-array wrapper, ROT transformation, AES-128-GCM encrypted payload blobs, and obfuscator.io-style string arrays
T1105	Ingress Tool Transfer	Bun runtime downloaded from GitHub/Bun infrastructure if missing
T1552.001	Unsecured Credentials: Credentials in Files	Searches `.env`, shell history, Docker config, SSH keys, cloud credential files, and local secrets
T1552.007	Unsecured Credentials: Container and Cloud Instance Metadata API	Queries AWS, Azure, and GCP metadata endpoints for temporary credentials and identities
T1552.003	Unsecured Credentials: Bash History	Shell history collection reported as part of local developer-secret harvesting
T1528	Steal Application Access Token	GitHub, npm, cloud, Vault, Kubernetes, and GitHub Actions tokens targeted
T1611	Escape to Host	Public reporting describes container-based privilege-escalation attempts that bind-mount host sudoers paths in CI contexts
T1053.003	Scheduled Task/Job: Cron	Family-level persistence includes scheduled/user-service style recurring execution; validate per host
T1543.002	Create or Modify System Process: Systemd Service	Linux user service `kitty-monitor.service` and `gh-token-monitor.service`
T1543.001	Create or Modify System Process: Launch Agent	macOS `com.user.kitty-monitor.plist` and `com.user.gh-token-monitor.plist`
T1567.001	Exfiltration Over Web Service: Exfiltration to Code Repository	GitHub repositories used as dead-drop storage for encrypted result JSON
T1102.002	Web Service: Bidirectional Communication	GitHub API used for repository creation, commits, workflow modification, and exfiltration
T1485	Data Destruction	Token monitor can run destructive commands such as deleting home/Documents directories after token invalidation
T1070.004	Indicator Removal: File Deletion	Loader deletes transient payload file after Bun execution

8. Detection Engineering

8.1. Detection Logic Summary

Signal	Telemetry Source	Analytic Goal	Tuning Notes

Package install spawns Bun or downloads Bun	EDR process telemetry, CI logs	Detect install-time staging and runtime bootstrap	Tune for projects that legitimately use Bun; require parent `npm`/`node` and temporary path
`preinstall` hook added to unexpected packages	Package-lock diffs, registry metadata, SCA tooling	Identify lifecycle-script abuse	Baseline approved packages with install scripts
`binding.gyp` command expansion	Repository/dependency static scanning	Detect Phantom Gyp family variants	Look for `<!(node ...`, shell redirection, or hidden loader execution
GitHub repo description `Miasma: The Spreading Blight`	GitHub audit/API inventory	Find dead-drop repositories	Include public repos created by users, bots, and CI tokens
Anthropic-shaped exfil path	Proxy, DNS, EDR network telemetry	Detect suspicious `/v1/api` use from Node/Bun during package install	Do not block all Anthropic traffic without business impact review
Developer-tool persistence files	File integrity, EDR file events	Find re-execution hooks	Review `.claude`, `.vscode`, `.github`, `.cursor`, `.gemini`, MCP/Aider files
npm publishing anomalies	npm org audit, GitHub Actions logs	Detect worm propagation	Alert on unexpected patch releases, changed tarballs, added Bun dependency, and provenance from unusual workflows

8.2. Sigma: Bun Runtime During npm Install

```
title: Suspicious Bun Execution During npm Package Installation
id: 4f6f9b5f-9706-4a5f-8d18-miasma-npm-bun-install
status: experimental
description: Detects Node/npm package-install activity launching Bun or executing transient JavaScript
payloads, consistent with Miasma/Shai-Hulud staging behavior.
author: Lost Boys Cyber
date: 2026-06-16
references:
  - https://research.jfrog.com/post/shai-hulud-miasma-redhat-cloud-services/
  - https://www.microsoft.com/en-us/security/blog/2026/06/02/preinstall-persistence-inside-red-hat-npm-
miasma-credential-stealing-campaign/
logsource:
  category: process_creation
detection:
  selection_parent:
    ParentImage|endswith:
      - '/node'
      - '/npm'
      - '/pnpm'
      - '/yarn'
      - '\\node.exe'
      - '\\npm.cmd'
      - '\\pnpm.cmd'
      - '\\yarn.cmd'
  selection_bun:
    Image|endswith:
      - '/bun'
```

```
      - '\\bun.exe'
  selection_tmp_payload:
    CommandLine|contains:
      - 'bun run /tmp/p'
      - 'bun run "'/tmp/p'
      - 'bun run %TEMP%'
      - 'bun run C:\\Users\\'
  selection_download:
    CommandLine|contains:
      - 'bun-v1.3.13'
      - 'github.com/oven-sh/bun/releases/download'
      - 'curl -fsSL https://bun.sh/install'
  condition: selection_parent and (selection_bun or selection_tmp_payload or selection_download)
falsepositives:
  - Projects that legitimately install Bun as part of a build. Require correlation with npm install
timing, affected packages, or temporary payload paths.
level: high
```

8.3. YARA: Miasma Loader and Campaign Markers

```
rule LBC_Miasma_ShaiHulud_Npm_Loader_Markers
{
    meta:
        description = "Detects public Miasma/Shai-Hulud npm loader, dead-drop, and persistence markers"
        author = "Lost Boys Cyber"
        date = "2026-06-16"
        confidence = "medium"
        reference = "https://research.jfrog.com/post/shai-hulud-miasma-redhat-cloud-services/"
    strings:
        $campaign = "Miasma: The Spreading Blight" ascii wide
        $threat = "IfYouInvalidateThisTokenItWillNukeTheComputerOfTheOwner" ascii wide
        $path1 = "results/results-" ascii wide
        $path2 = "/tmp/.bun_ran" ascii wide
        $path3 = "gh-token-monitor" ascii wide
        $hook1 = ".claude/settings.json" ascii wide
        $hook2 = ".vscode/tasks.json" ascii wide
        $cmd1 = "bun run" ascii wide
        $cmd2 = "gh auth token" ascii wide
        $anthropic = "api.anthropic.com" ascii wide
        $oidc1 = "ACTIONS_ID_TOKEN_REQUEST_TOKEN" ascii wide
        $oidc2 = "OIDC_PACKAGES" ascii wide
    condition:
        filesize < 10MB and (
            $campaign or $threat or
            (2 of ($path*) and any of ($cmd*)) or
            ($anthropic and $cmd1) or
            (all of ($oidc*) and $cmd1) or
            (2 of ($hook*) and $cmd1)
        )
}
```

8.4. KQL: Endpoint Process Detection

```
// Miasma/Shai-Hulud: npm install spawning Bun or transient payload execution
DeviceProcessEvents
| where Timestamp > ago(30d)
| where FileName in~ ("bun", "bun.exe", "curl", "curl.exe", "unzip", "unzip.exe", "node", "node.exe")
| extend Parent = tostring(InitiatingProcessFileName), Cmd = tostring(ProcessCommandLine), ParentCmd =
tostring(InitiatingProcessCommandLine)
| where Parent in~ ("node", "node.exe", "npm", "npm.cmd", "pnpm", "pnpm.cmd", "yarn", "yarn.cmd",
"npx", "npx.cmd")
   or ParentCmd has_any ("npm install", "npm ci", "pnpm install", "yarn install", "preinstall", "node
index.js")
| where Cmd has_any ("bun run", "/tmp/p", "bun-v1.3.13", "github.com/oven-sh/bun/releases/download",
"curl -fsSL https://bun.sh/install", "node index.js")
| project Timestamp, DeviceName, AccountName, FileName, Cmd, Parent, ParentCmd, FolderPath,
InitiatingProcessFolderPath
```

```
| order by Timestamp desc
```

8.5. KQL: Persistence File Creation

```
// Miasma/Shai-Hulud developer-tool and token-monitor persistence artifacts
DeviceFileEvents
| where Timestamp > ago(30d)
| extend Path = strcat(FolderPath, "/", FileName)
| where Path has_any (
    ".claude/settings.json", ".claude/setup.mjs", ".vscode/tasks.json", ".github/setup.js",
    ".config/index.js", "kitty-monitor.service", "gh-token-monitor", "com.user.kitty-monitor.plist",
    "com.user.gh-token-monitor.plist", ".cursor/rules/setup.mdc", ".gemini/settings.json",
    ".cursorrules", ".windsurfrules", "mcp.json", ".aider.conf.yml")
| project Timestamp, DeviceName, AccountName, ActionType, Path, InitiatingProcessFileName,
InitiatingProcessCommandLine
| order by Timestamp desc
```

8.6. KQL: Suspicious Anthropic-Shaped Exfiltration During Build

```
// Hunt full request/path context where available; fallback to process/domain correlation.
DeviceNetworkEvents
| where Timestamp > ago(30d)
| where RemoteUrl has "api.anthropic.com" or RemoteUrl has "anthropic"
| where InitiatingProcessFileName in~ ("node", "node.exe", "bun", "bun.exe", "npm", "npm.cmd")
| where InitiatingProcessCommandLine has_any ("npm", "preinstall", "node index.js", "bun run",
"/tmp/p")
| project Timestamp, DeviceName, AccountName, RemoteUrl, RemoteIP, RemotePort,
InitiatingProcessFileName, InitiatingProcessCommandLine
| order by Timestamp desc
```

8.7. SPL: GitHub Dead-Drop and npm Propagation Markers

```
index=github_audit OR index=proxy OR index=edr
("Miasma: The Spreading Blight" OR "results/results-" OR
"IfYouInvalidateThisTokenItWillNukeTheComputerOfTheOwner" OR "OIDC_PACKAGES" OR "bun run _index.js" OR
"bun run $GITHUB_ACTION_PATH/index.js")
| table _time index sourcetype user actor repo action src_ip dest url file_path command message
| sort - _time
```

8.8. Detection Validation and Blind Spots

These detections are designed for high-signal correlation, not standalone blocking. Bun is legitimate in many JavaScript environments; `api.anthropic.com` may be legitimate in organizations using AI tools; and GitHub repository creation is normal for developers. Confidence rises sharply when signals combine: npm install parentage, temporary `/tmp/p*.js` payloads, Bun bootstrap downloads, affected package names, Miasma repo descriptions, developer-tool persistence files, or sudden package-publishing changes. Blind spots include ephemeral CI runners without process telemetry, private package registries that do not retain tarball diffs, and cloud environments where metadata API access is not logged centrally.

9. Threat Hunting

9.1. Hunt Hypotheses

- A developer endpoint or CI runner installed a malicious `@redhat-cloud-services` package and spawned Node/Bun processes during package installation.
- A compromised identity created public GitHub repositories with Miasma dead-drop markers or committed `results/results-*.json` files.
- The malware added persistence through Claude/AI-tool hooks, VS Code tasks, GitHub workflow files, user services, or token-monitor artifacts.
- A stolen npm or OIDC publishing identity republished packages with added `preinstall` scripts, `bun` dependencies, or suspicious patch-version bumps.
- Cloud credentials were accessed from a build host immediately after package installation, visible as metadata-service queries or secret-manager enumeration.

9.2. Hunt Query: Dependency Lockfile Exposure

```
# Run from source-code roots and artifact caches to identify known affected package/version pairs.
python3 - <<'PY'
import json, pathlib
known = {
  "@redhat-cloud-services/types": {"3.6.1", "3.6.2", "3.6.4"},
  "@redhat-cloud-services/frontend-components": {"7.7.2", "7.7.3", "7.7.5"},
  "@redhat-cloud-services/rbac-client": {"9.0.3", "9.0.4", "9.0.6"},
  "@redhat-cloud-services/sources-client": {"3.0.10", "3.0.11", "3.0.13"},
  "@redhat-cloud-services/vulnerabilities-client": {"2.1.8", "2.1.9", "2.1.11"},
}
for path in pathlib.Path('.').rglob('package-lock.json'):
    try:
        data = json.loads(path.read_text(errors='ignore'))
    except Exception:
        continue
    packages = data.get('packages', {})
    for meta in packages.values():
        name, ver = meta.get('name'), meta.get('version')
        if name in known and ver in known[name]:
            print(f"{path}: {name}@{ver}")
PY
```

9.3. Hunt Query: GitHub Audit for Dead Drops and Workflow Tampering

```
// Adapt table names to GitHub audit ingestion. Looks for Miasma repo markers and suspicious
workflow/package commits.
GitHubAuditLog_CL
| where TimeGenerated > ago(45d)
| where tostring(Action) has_any ("repo.create", "git.push", "workflow", "actions", "artifact",
"release")
   or tostring(Repo) has_any ("miasma", "hades")
   or tostring(Details) has_any ("Miasma: The Spreading Blight", "results/results-",
"IfYouInvalidateThisTokenItWillNukeTheComputerOfTheOwner", "bun run _index.js", "OIDC_PACKAGES",
"format-results")
| project TimeGenerated, Actor=tostring(Actor), Action=tostring(Action), Repo=tostring(Repo),
Details=tostring(Details), IPAddress=tostring(IPAddress)
| order by TimeGenerated desc
```

9.4. Hunt Query: Cloud Metadata Access from Build Processes

```
// Detect package-install lineage reaching cloud metadata services.
DeviceNetworkEvents
| where Timestamp > ago(30d)
| where RemoteIP in ("169.254.169.254") or RemoteUrl has_any ("metadata.google.internal",
"management.azure.com", "graph.microsoft.com", "vault.azure.net")
| where InitiatingProcessFileName in~ ("node", "node.exe", "bun", "bun.exe", "curl", "curl.exe",
"python", "python3")
| where InitiatingProcessCommandLine has_any ("npm", "node index.js", "bun run", "preinstall",
"/tmp/p")
| project Timestamp, DeviceName, AccountName, RemoteIP, RemoteUrl, InitiatingProcessFileName,
InitiatingProcessCommandLine
| order by Timestamp desc
```

9.5. Triage Guidance for Hunt Hits

A meaningful hit is one that ties at least two phases together: affected package/version plus install-time Node/Bun execution; Bun staging plus credential-store reads; GitHub repo creation plus `results/results-*.json`; npm publish events plus added lifecycle hooks; or persistence artifacts plus Miasma markers. Benign lookalikes include legitimate Bun adoption, normal Anthropic API traffic from approved AI tooling, and routine GitHub workflow edits. Escalate to incident response when a hit occurs on a CI runner, release engineer workstation, package maintainer account, or any system with cloud deployment permissions.

10. Recommendations

Priority	Owner	Action	Rationale
Critical	DevOps / Platform	Inventory lockfiles, package caches, CI logs, and artifact builds for affected `@redhat-cloud-services` versions from 2026-06-01 onward	Establish exposure before rotating credentials or rebuilding releases
Critical	Incident Response	Isolate affected endpoints/runners before revoking GitHub/npm/cloud tokens	Prevent token-monitor destructive behavior and preserve evidence
Critical	Identity / Cloud	Rotate GitHub, npm, AWS, Azure, GCP, Vault, Kubernetes, SSH, Docker, and CI/CD secrets exposed to affected systems from clean workstations	Malware targeted these secret classes directly
High	AppSec / Supply Chain	Block or require review for package lifecycle scripts in CI; use `npm ci --ignore-scripts` where feasible and explicitly allow required scripts	Reduces install-time execution risk
High	DevOps	Enforce ephemeral, least-privilege CI runners with short-lived scoped tokens and no broad package-publishing authority	Limits worm propagation and credential blast radius
High	GitHub / SCM Admins	Audit public/private repositories for Miasma descriptions, `results/` files, unexpected workflows, action conversions, and suspicious signed commits	Finds exfiltration and propagation artifacts
High	Package Registry Admins	Review npm org publishing events, package tarball diffs, patch-version bumps, added `preinstall` hooks, added Bun dependencies, and unexpected trusted-publishing provenance	Detects republished poisoned packages
Medium	Detection Engineering	Deploy process, file, network, GitHub, npm, and cloud metadata detections from Section 8 with local tuning	Provides coverage across endpoint and CI telemetry
Medium	Developer Experience	Educate developers that deleting `node_modules` is not sufficient; require persistence-file review after exposure	Miasma persists outside dependency folders

Medium	Governance	Treat SLSA/provenance as one signal rather than proof of clean artifacts; pair attestations with behavioral and content inspection	Miasma abused legitimate OIDC publishing paths to create valid-looking attestations

Short-term containment should prioritize isolation, evidence collection, and credential rotation. Long-term control improvements should focus on dependency-execution governance, build-time egress controls, package-diff review, secret minimization in CI, and supply-chain anomaly monitoring. Organizations using AI coding assistants should include assistant configuration files and repository instruction/rule files in integrity monitoring because this malware family explicitly targets developer-tool execution paths.

11. References

- [1] Microsoft Threat Intelligence. “Preinstall to persistence: Inside the Red Hat npm Miasma credential-stealing campaign.” Microsoft Security Blog, 2026-06-02 / RSS published 2026-06-03. https://www.microsoft.com/en-us/security/blog/2026/06/02/preinstall-persistence-inside-red-hat-npm-miasma-credential-stealing-campaign/
- [2] Guy Korolevski, JFrog Security Research. “Shai-Hulud - Miasma: The Spreading Blight Hits Red Hat npm Packages.” 2026-06-01, updated 2026-06-04 and 2026-06-07. https://research.jfrog.com/post/shai-hulud-miasma-redhat-cloud-services/
- [3] Cloud Security Alliance. “Miasma: Red Hat npm Supply Chain Worm.” Research note, 2026-06-03. https://labs.cloudsecurityalliance.org/wp-content/uploads/2026/06/CSA_research_note_miasma_npm_supply_chain_redhat_20260603-csa-styled.pdf
- [4] StepSecurity. “Miasma npm Supply Chain Attack: Self-Spreading Worm via Phantom Gyp.” 2026-06-03. https://www.stepsecurity.io/blog/binding-gyp-npm-supply-chain-attack-spreads-like-worm
- [5] Ravie Lakshmanan, The Hacker News. “Miasma Supply Chain Attack Compromises Red Hat npm Packages with Credential-Stealing Worm.” 2026-06-01. https://thehackernews.com/2026/06/miasma-supply-chain-attack-compromises.html
- [6] Red Hat Product Security. “RHSB-2026-006 Supply chain compromise of @redhat-cloud-services npm packages.” 2026-06-01. Referenced by CSA and public reporting; use vendor portal for current customer guidance.
- [7] SafeDep, Wiz, Socket, Aikido Security, OX Security, ReversingLabs, and other public researchers cited by CSA/THN/JFrog reporting. Use these as corroborating sources for package exposure and variant-specific details when expanding the IOC corpus.